

重大 AI 更新提醒

10-09 | llama.cpp 同日三连发：benchmark 参数修正、CUDA 数值一致性修复与 SSM_SCAN 冗余拷贝消除

本期导读

本期聚焦 ggml-org/llama.cpp 在 2026-10-09 连续发布的三个版本。b11535 修正 llama-bench 对 fitc 参数的处理逻辑，b11538 修复 MSVC 下 CUDA 舍入问题使 CPU 与 GPU 结果一致，b11534 则针对 SSM_SCAN 路径移除冗余 CUDA 拷贝。三条更新分别覆盖评测工具链、数值正确性与推理性能，对本地部署与 CUDA 后端开发者具有直接参考价值。

已检查来源

GitHub Release

1. llama.cpp b11535：llama-bench 在 fitc 大于所需基准尺寸时正确遵循该参数

来源 GitHub Release | 类型 GitHub Release | 主题 开发者生态与数据工具 | 时间 10-09 13:49 | 分数 8

是什么 这是 ggml-org/llama.cpp 的官方 GitHub Release b11535，核心变更是让 llama-bench 工具在用户传入的 fitc (fit context) 值大于实际基准测试所需尺寸时，仍然尊重该设置，而不是被内部逻辑覆盖。该版本同时提供 macOS Apple Silicon (arm64)、macOS Intel (x64)、iOS XCFramework、Ubuntu x64/arm64 等平台的预编译二进制包。

通俗解释 llama-bench 是 llama.cpp 自带的性能跑分工具，用来测量模型在特定上下文长度下的推理速度。可以把它想象成给汽车测油耗：你告诉它按 500 公里路程来测，但工具内部可能觉得其实 300 公里就够了，于是擅自改成 300 公里，导致测出来的油耗和你关心的长途场景对不上。fitc 就是那个路程参数。b11535 修复后，只要你指定的 fitc 比工具默认需要的更大，它就会老老实实按你的要求跑，这样得到的 benchmark 数据才真正反映你实际部署时的长上下文负载。

主要作用 解决 llama-bench 在长上下文基准测试中不尊重用户指定 fitc 参数的问题，避免评测结果与实际部署场景脱节。对于需要评估模型在长上下文下显存占用、吞吐和延迟的团队，这一修复让 benchmark 数据更可信，减少因工具内部截断而导致的误判。

核心功能 1) llama-bench 在 fitc 大于所需基准尺寸时按用户值执行，不再被内部逻辑覆盖；2) 覆盖 macOS Apple Silicon、macOS Intel、iOS XCFramework、Ubuntu x64/arm64 等多平台预编译包；3) 与 llama.cpp 主仓库同步发布，便于直接替换二进制验证；4) 变更关联 PR #28331，可追溯具体实现。

对比判断 候选摘要未提供与旧版本或其他 benchmark 工具的速度、成本对比数据，暂无可信公开对比数据。

适合谁 适合使用 llama.cpp 做本地模型评测的开发者、需要在长上下文场景下做容量规划的推理工程师，以及维护 CI 基准测试、关注 benchmark 可复现性的团队。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11535>

2. llama.cpp b11538：修复 CUDA 舍入问题，MSVC 下 CPU 与 GPU 结果一致

来源 GitHub Release | 类型 GitHub Release | 主题 推理性能与基础设施 | 时间 10-09 15:11 | 分数 7

是什么	这是 ggml-org/llama.cpp 的官方 GitHub Release b11538，核心变更是修复 CUDA 后端的舍入（round）问题，使得在 MSVC（Microsoft Visual C++）编译环境下，CPU 与 GPU 的计算结果保持一致。该版本同样附带 macOS、iOS、Linux 多平台预编译二进制，变更关联 PR #30229。
通俗解释	在本地跑大模型时，同一份输入交给 CPU 和 GPU 计算，理论上应该得到几乎一样的结果。但浮点数运算存在舍入差异：就像两个人用不同计算器算 $1 \div 3$ ，一个显示 0.333，另一个显示 0.334，单次差别很小，可在多层神经网络里会逐层放大，最终导致 CPU 和 GPU 输出不一致。b11538 修复了 CUDA 路径上的舍入处理，让 MSVC 编译出的版本在 CPU 和 GPU 之间 对得上账。这对调试、复现实验和验证模型正确性非常关键。
主要作用	解决 MSVC 环境下 CUDA 后端与 CPU 后端计算结果不一致的问题，提升跨设备推理的数值一致性和可复现性。对于在 Windows 上用 MSVC 构建 llama.cpp 并同时使用 CPU/GPU 的开发者，这一修复能减少因舍入差异导致的输出漂移和调试困扰。
核心功能	1) 修复 CUDA 舍入问题，使 MSVC 下 CPU 与 GPU 结果一致；2) 覆盖 macOS Apple Silicon、macOS Intel、iOS XCFramework、Ubuntu x64/arm64/s390x 等多平台；3) 关联 PR #30229，便于查看具体改动；4) 与主仓库同步发布，可直接替换二进制验证一致性。
对比判断	候选摘要未提供修复前后的数值误差幅度、速度或成本对比数据，暂无可信公开对比数据。
适合谁	适合在 Windows/MSVC 环境下构建 llama.cpp 并使用 CUDA 后端的开发者、需要 CPU/GPU 结果一致性的推理工程师，以及做数值正确性验证和回归测试的研究人员。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11538>

3. llama.cpp b11534 : CUDA 移除 SSM_SCAN 后的冗余拷贝，并将状态快照融合进循环缓存

来源 GitHub Release | 类型 GitHub Release | 主题 推理性能与基础设施 | 时间 10-09 13:29 | 分数 7

是什么	这是 ggml-org/llama.cpp 的官方 GitHub Release b11534，核心变更是在 CUDA 后端中移除 SSM_SCAN 之后的冗余 CUDA 拷贝，并把更新后的状态快照拷贝融合进 recurrent cache；同时在 K==1（非投机解码）场景下也移除冗余 CUDA 拷贝。变更关联 PR #29807，附带多平台预编译二进制。
通俗解释	SSM_SCAN 是处理状态空间模型（SSM）类架构时的一种扫描计算，常用于 Mamba 等非 Transformer 结构。计算过程中会产生中间状态，需要从一块显存搬到另一块显存保存。原来的做法是先算完，再单独搬一次，就像做完菜再单独装盘；b11534 把装盘动作直接融进做菜流程里，省掉一次多余的搬运。在 K==1（即不做投机解码）的普通推理场景下也做了同样优化。显存搬运虽然不复杂，但次数多了会拖慢速度，减少冗余拷贝有助于提升推理效率。
主要作用	解决 CUDA 后端在 SSM_SCAN 之后存在冗余显存拷贝、拖慢推理的问题，通过融合拷贝与移除冗余操作来降低开销。对于使用 Mamba 等 SSM 架构模型、在 CUDA 上做本地推理的开发者，这一优化直接关系到吞吐和延迟表现。
核心功能	1) 将更新后状态快照的拷贝融合进 recurrent cache，与 ssm_scan 合并执行；2) 移除 SSM_SCAN 之后的冗余 CUDA 拷贝；3) 在 K==1（非投机解码）场景下同样移除冗余拷贝；4) 覆盖 macOS、iOS、Linux 多平台预编译包，关联 PR #29807。
对比判断	候选摘要未提供优化前后的速度、显存占用或成本对比数据，暂无可信公开对比数据。
适合谁	适合在 CUDA 上部署 Mamba/SSM 类模型的推理工程师、关注显存拷贝开销与 kernel 融合的底层优化开发者，以及维护 llama.cpp CUDA 后端的贡献者。
直达链接	https://github.com/ggml-org/llama.cpp/releases/tag/b11534

以上内容均基于候选中的 GitHub Release 信息整理，未引入外部数据；具体性能收益请以实际环境测试为准。